Development Guidelines

Guia de Desenvolvimento e Submissão

Índice

1. Objetivo do Projeto 2
2. Regras de Avaliação do Projeto 3
2.1. Componente de avaliação 3
2.2. Requisitos obrigatórios para apresentação e defesa 3
2.3. Entrega do Projeto 4
2.4. Situações que conduzem à classificação de 0 valores 4
2.5. Penalizações aplicáveis 4
2.6 Controlo de Versões (Git) 5
3. Regras de Desenvolvimento 7
3.1 Regras de Desenvolvimento do Frontend 7
3.2 Regras de Desenvolvimento da Base de Dados 11
3.3 Regras de Desenvolvimento do Backend 11
3.4 Estrutura de Diretórios e Organização do Projeto 11
3.5 Aspetos valorizados no desenvolvimento 12

Este documento define a estrutura geral do projeto, bem como as regras de desenvolvimento e avaliação. Algumas secções serão detalhadas progressivamente ao longo do semestre, de acordo com os conteúdos lecionados na unidade curricular (Frontend, Bases de Dados e Backend).

A leitura integral deste documento é obrigatória.
A submissão do projeto pressupõe o conhecimento e a aceitação integral das regras nele definidas, não podendo o seu desconhecimento ser invocado como justificação para o incumprimento das mesmas.

1. Objetivo do Projeto

Cada estudante deverá desenvolver **individualmente** uma aplicação web que simule um **sistema de gestão do inventário hospitalar de equipamentos médicos**.
O sistema deverá permitir registar e gerir informação relevante sobre equipamentos, fornecedores e outros elementos associados ao funcionamento de um inventário hospitalar.
O objetivo não consiste apenas em criar uma lista de equipamentos, mas sim em desenvolver uma **estrutura de aplicação web** organizada, que possa servir de base à futura implementação de um sistema de gestão de manutenção de equipamentos médicos.
O projeto será **desenvolvido progressivamente ao longo do semestre**, acompanhando os conteúdos lecionados nas aulas:

- **Frontend (HTML, CSS, Bootstrap e JavaScript)**
- **Bases de Dados (MySQL)**
- **Backend (PHP)**

Algumas regras de desenvolvimento serão detalhadas à medida que cada módulo for introduzido.

2. Regras de Avaliação do Projeto

2.1. Componente de avaliação

O projeto corresponde à **componente M1-NR — Projeto**, com peso de **60% na classificação final da unidade curricular.**

A classificação final da unidade curricular é calculada de acordo com a fórmula definida na Ficha da Unidade Curricular:

$$CF = M1 \times 0.6 + EG \times 0.4$$

onde:

- M1 — Classificação da componente **Projeto (M1-NR)**
- EG — Classificação do **Exame Global**

Nota mínima exigida em cada componente**: 8,5 valores.**

2.2. Requisitos obrigatórios para apresentação e defesa

Para que o estudante possa realizar a **apresentação e defesa do projeto**, deverão estar previamente cumpridos os seguintes requisitos:

- Submissão do **código do projeto na plataforma Moodle**, dentro do prazo definido.
- Entrega do **relatório técnico individual**.
- Entrega da **declaração de utilização de Inteligência Artificial**, devidamente preenchida e assinada.

A apresentação e defesa do trabalho constituem requisito obrigatório de avaliação, sendo condição necessária para a atribuição de classificação. A não comparência à apresentação e defesa do projeto, sem motivo devidamente justificado nos termos regulamentares da instituição, implica a **atribuição da classificação de 0 (zero valo**res) na componente M1-NR — Projeto, mesmo que o trabalho tenha sido previamente submetido, **não sendo possível proceder à sua avaliação posterior.**

Durante a apresentação oral, cada estudante deverá demonstrar **domínio integral do trabalho** submetido, sendo capaz de:

- Explicar a estrutura da aplicação desenvolvida
- Justificar as decisões técnicas tomadas
- Demonstrar o funcionamento das principais funcionalidades
- Responder às questões colocadas sobre o código e o desenvolvimento do projeto

2.3. Entrega do Projeto

A entrega do projeto deverá ocorrer **até à data e hora definidas.**

▪ A submissão efetuada **até 30 minutos após o termo do prazo** implicará uma **penalização de 25% na classificação da componente M1-NR — Projeto.**

▪ Ultrapassado esse período, **a entrega não será aceite** e será atribuída **classificação de 0 (zero valores) na componente M1-NR — Projeto.**

2.4. Situações que conduzem à classificação de 0 valores

Será atribuída a classificação de **0 (zero valores)** na componente M1-NR — Projeto nas seguintes situações:

- **Não entrega da declaração de utilização de Inteligência Artificial,** devidamente preenchida e assinada.
- **Deteção de plágio,** designadamente a submissão de código total ou parcialmente produzido por terceiros ou copiado de outras fontes sem referência adequada.
- **Não entrega do trabalho dentro do prazo definido**.
- **Não comparência à apresentação e defesa do projeto**, sem motivo devidamente justificado nos termos regulamentares da instituição.
- **Incapacidade de explicar ou demonstrar domínio do trabalho** submetido durante a apresentação e defesa oral.

2.5. Penalizações aplicáveis

Serão aplicadas penalizações na **classificação da componente M1-NR** — Projeto nas seguintes situações:

- **Não utilização de controlo de versões com Git**, conforme definido nas regras do projeto**: penalização de 25% na classificação da componente M1-NR — Projeto**.

- **Número de commits inferior ao mínimo definido nas regras de desenvolvimento**: penalização de até **25%,** a definir pelo docente em função da gravidade da situação, **na classificação da componente M1-NR — Projeto**.

- **Commits realizados apenas no final do semestre, sem evolução ao longo do tempo**: o histórico de commits **não será considerado evidência de desenvolvimento progressivo do projeto**, podendo resultar numa **penalização de até 25%,** a definir pelo docente em função da gravidade da situação, **na classificação da componente M1-NR — Projeto**.

O histórico de commits poderá ser analisado como evidência do processo de desenvolvimento do trabalho ao longo do semestre.

O docente poderá solicitar ao estudante que explique qualquer parte do código apresentado. A incapacidade de o fazer poderá ser interpretada como **ausência de domínio do trabalho submetido**.

2.6 Controlo de Versões (Git)

Durante o desenvolvimento do projeto, o estudante deverá utilizar **Git** para realizar o controlo de versões do código.
O repositório **deverá** refletir a **evolução real do desenvolvimento do projeto ao longo do semestre**.

- **Configuração obrigatória do Git**

A configuração do Git deve utilizar o nome e o email institucional do estudante, garantindo a identificação individual da autoria dos commits.

Exemplo de configuração:

```
git config --global user.name "Nome do Estudante"
git config --global user.email "email@isep.ipp.pt"
```

Cada estudante deverá utilizar **uma configuração pessoal e única**. Commits realizados com identidades genéricas, partilhadas ou não identificáveis poderão **não ser considerados válidos para efeitos de avaliação**.

- **Regras**

  - Devem ser realizados **commits regulares ao longo do desenvolvimento do projeto,** refletindo a evolução do trabalho.
  - Cada commit deverá representar **uma alteração significativa ou funcionalidade implementada.**
  - Commits com mensagens como "*teste*", "*update*" ou semelhantes **não serão considerados válidos.**
  - O histórico de commits deverá evidenciar um **desenvolvimento progressivo ao longo do tempo**, não sendo considerados válidos históricos com submissões concentradas num único dia.

- **Registo do histórico de commits**

O ficheiro (*commits.txt*) de histórico de commits deverá incluir informação **sobre data, hora e descrição dos commits**, sendo gerado através do seguinte comando:

```
git log --pretty=format:"%h - %an - %ad - %s" --date=iso > commits.txt
```

O ficheiro deverá ser incluído na pasta do projeto submetido.

- **Número mínimo de commits**

Para demonstrar desenvolvimento progressivo, o projeto deverá conter **no mínimo**:

- Frontend: **15 commits mínimos**
- Backend (PHP): **15 commits mínimos**
- Total mínimo exigido: **30 commits**

No caso de o aluno já ter iniciado o desenvolvimento do projeto antes da publicação do presente documento, serão considerados apenas os commits realizados a **partir da data de publicação**. Adicionalmente, **em função do grau de evolução já atingido**, poderá ser dispensado o cumprimento do mínimo de 15 commits relativos ao frontend, mediante análise

- **Exemplos de commits aceitáveis (a título exemplificativo)**

```
git commit -m "Estrutura inicial das páginas HTML"
git commit -m "Implementação da página de listagem de equipamentos"
git commit -m "Criação do formulário de registo de fornecedor"
git commit -m "Integração de validação JavaScript no formulário"
git commit -m "Implementação da ligação à base de dados"
```

3. Regras de Desenvolvimento

3.1 Regras de Desenvolvimento do Frontend

O desenvolvimento do **frontend** da aplicação web deve respeitar os princípios, tecnologias e boas práticas abordadas nas aulas, garantindo coerência visual, organização estrutural e qualidade da experiência de utilização.

3.1.1. Tecnologias obrigatórias

O frontend deve ser desenvolvido com recurso às seguintes tecnologias:

- HTML5, para a estruturação semântica das páginas;
- CSS3, para a definição da apresentação visual;
- JavaScript, para a implementação de interatividade;
- Bootstrap (versão mais recente), para construção de interfaces responsivas e consistentes.

Poderão ainda ser utilizados recursos complementares, desde que adequadamente integrados, tais como:

- Font Awesome (ícones);
- Chart.js (gráficos);
- DataTables.js (tabelas interativas);
- outras bibliotecas compatíveis com os objetivos do projeto.

3.1.2. Estrutura geral da aplicação

A aplicação deverá estar organizada em duas áreas principais:

- Área pública — acessível sem autenticação;
- Área privada — acessível após autenticação ou simulação de login.

3.1.3. Consistência visual

A aplicação deve apresentar um layout uniforme e coerente em todas as páginas.

Devem manter-se consistentes:

- cores;
- tipografia;
- espaçamentos;
- componentes visuais;
- estrutura de navegação.

O objetivo é garantir uma experiência visual homogénea e profissional.

3.1.4. Ficheiros individuais de CSS e JavaScript

Cada estudante deverá criar ficheiros próprios de estilos e scripts, identificados com o seu número de aluno.

Exemplo:

- assets/css/1940266.css
- assets/js/1940266.js

Estes ficheiros deverão conter exclusivamente:

- estilos utilizados nas páginas desenvolvidas pelo próprio estudante;
- scripts associados às funcionalidades por si implementadas.

3.1.5. Área pública

A área pública deverá incluir pelo menos uma página acessível sem login. Os conteúdos apresentados devem ser realistas e adequados ao contexto da aplicação, refletindo práticas plausíveis em Portugal (nomes, entidades, contactos, horários, serviços, etc.).

A área pública poderá assumir:

- uma única página com navegação por âncoras; ou
- várias páginas simples interligadas.

3.1.6. Gestão de conteúdos da área pública

Os conteúdos da área pública **devem obrigatoriamente** poder ser geridos através da área reservada, evitando a edição direta do HTML.

A aplicação deverá incluir mecanismos que permitam a atualização dinâmica dos conteúdos, garantindo separação entre apresentação e gestão da informação.

Exemplos de funcionalidades a implementar:

- edição de textos;
- atualização de secções;
- alteração de contactos;
- gestão de conteúdos informativos.

3.1.7. Área privada

Após autenticação (real ou simulada), o utilizador deverá ter acesso à área reservada da aplicação. Cada estudante deverá desenvolver uma aplicação funcional completa, correspondente a um sistema de gestão de inventário hospitalar de equipamentos médicos.

Componentes obrigatórios da área privada:

Cada estudante deverá implementar, no mínimo, os seguintes componentes:

- **Módulo de gestão (CRUD)**

O módulo de gestão de dados deverá ser desenvolvido de acordo com o **guião do projet**o, onde se encontram definidos:

- os dados a gerir;
- as entidades envolvidas;
- os requisitos funcionais;
- as operações obrigatórias.

O estudante deverá implementar as funcionalidades de criação, listagem, edição e eliminação de dados, conforme especificado nesse documento.

- **Dashboard de gestão**

Interface com visualização de dados através de:

- gráficos; ou
- indicadores estatísticos.

Os elementos apresentados devem estar relacionados com os dados registados na aplicação, como por exemplo:

- número total de equipamentos;
- equipamentos por categoria;
- equipamentos por localização;
- equipamentos ativos/inativos.

Podem ser utilizadas bibliotecas como:

- Chart.js
- Google Charts

- **Backoffice da área pública**

Interface administrativa que permita gerir os conteúdos da área pública da aplicação.

Deverá permitir, nomeadamente:

- alteração de textos;
- atualização de conteúdos informativos;
- gestão de elementos apresentados ao utilizador.

3.1.8. Responsividade

A aplicação deve adaptar-se corretamente a diferentes dispositivos:

- computadores;
- tablets;
- telemóveis.

Deverão ser utilizados os mecanismos de responsividade do Bootstrap, garantindo legibilidade e boa organização em qualquer dimensão de ecrã.

3.1.9. Usabilidade e acessibilidade

A aplicação deverá respeitar princípios básicos de usabilidade e acessibilidade.

Devem ser asseguradas as seguintes boas práticas:

- contraste visual adequado;
- utilização de etiquetas (label) nos formulários;
- identificação clara de campos obrigatórios;
- mensagens de erro claras e específicas;
- destaque de botões e elementos interativos;
- navegação intuitiva;
- indicação visual de estados (ativo, selecionado, erro, etc.).

O objetivo é garantir uma utilização simples, clara e acessível

3.1.10. Utilização de Bootstrap

É obrigatória a utilização de componentes do Bootstrap ao longo da aplicação. Deverá ser incluído pelo menos um componente não explorado nas aulas práticas.

Exemplos:

- Accordion
- Carousel
- Offcanvas
- Toasts
- Progress Bars
- Tooltips

A sua utilização deverá ser funcional e coerente com a aplicação.

3.2 Regras de Desenvolvimento da Base de Dados

As regras de modelação da base de dados, incluindo normalização, definição de tabelas, chaves primárias, chaves estrangeiras e relacionamentos, serão apresentadas durante o módulo de Bases de Dados.

Após a lecionação desse módulo, este guia será atualizado com os requisitos técnicos específicos a cumprir.

3.3 Regras de Desenvolvimento do Backend

As regras relativas ao desenvolvimento do backend, incluindo:

- organização do código PHP;
- processamento de formulários;
- operações CRUD;
- ligação à base de dados;
- gestão de sessões e autenticação;
- validação de dados;

serão apresentadas durante o módulo de Backend. Este documento será posteriormente atualizado com essas orientações.

3.4 Estrutura de Diretórios e Organização do Projeto

A estrutura do projeto deve ser organizada de forma clara, coerente e modular, respeitando as boas práticas abordadas nas aulas.
A aplicação deverá incluir, no mínimo, a seguinte estrutura:

```
/nomedoprojeto/
├── public/
├── private/
├── assets/
│   ├── css/
│   ├── js/
│   └── img/
│   └── . . .
├── README.md
```

Regras obrigatórias

- A pasta public/ deve conter as páginas acessíveis sem autenticação.
- A pasta private/ deve conter as páginas da área reservada.
- A pasta assets/ deve agrupar os recursos estáticos da aplicação, nomeadamente:
  - css/ — ficheiros de estilos;
  - js/ — ficheiros JavaScript;
  - img/ — imagens utilizadas na aplicação
- O ficheiro README.md deve descrever a organização do projeto, identificando a estrutura de diretórios e os autores.

A organização em assets/ segue as práticas adotadas nas aulas, promovendo a separação entre conteúdo (HTML) e recursos estáticos (CSS, JavaScript e imagens).

3.5 Aspetos valorizados no desenvolvimento

Serão valorizados os seguintes aspetos:

- correta aplicação dos conteúdos lecionados;
- organização e clareza da estrutura do projeto;
- coerência visual e funcional;
- utilização adequada de bibliotecas e componentes;
- exploração autónoma de novas soluções;
- criatividade e originalidade.

A utilização de tecnologias ou abordagens não exploradas nas aulas será valorizada, desde que devidamente integrada e justificada no relatório técnico. Será valorizada a coerência global da aplicação, bem como a integração entre as diferentes componentes desenvolvidas.